UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

JUEGO UNO

Materia: Interfaces gráficas con aplicaciones

Integrantes

Gisela Rodríguez Gerónimo

Alejandra Martínez Arévalo

Fecha de entrega: 8 de noviembre de 2026

1. Descripción general del proyecto

El proyecto consiste en desarrollar una versión del juego de cartas UNO, programada en C#. En su primera versión, el juego se ejecutaba en una sola computadora, por lo que una persona controlaba las acciones de todos los jugadores. La aplicación incluye una interfaz visual con cartas, botones y elementos gráficos para facilitar el desarrollo de las partidas.

Para almacenar la información del juego se utiliza una base de datos MySQL, conectada a través de una API desarrollada en Python con FastAPI. Esta API permite que la aplicación envíe información de jugadores, partidas y movimientos, para registrar el historial y el log del juego.

Entre las funciones implementadas se encuentran las clases para representar las cartas, el mazo y los jugadores; el reparto de cartas y la visualización de las manos iniciales; efectos de cartas como robar, saltar turno y reversa; la interfaz gráfica; y el registro de movimientos e historial de partidas en la base de datos.

El objetivo es integrar la lógica del juego, la interfaz y el almacenamiento de datos en una aplicación funcional que permita jugar y consultar la información registrada.

En la segunda etapa, el juego se convirtió en una aplicación cliente-servidor conectada por red mediante sockets TCP. Una computadora funciona como servidor, donde corren la API, la base de datos y el servidor del juego en Python. Cada jugadora abre el juego en su propia laptop y se conecta escribiendo la IP del servidor.

Ahora el servidor controla la partida: baraja y reparte las cartas, decide de quién es el turno, valida cada jugada, aplica los efectos de las cartas especiales, lleva el temporizador para cantar UNO y detecta a la ganadora. Cuando una jugadora tira una carta, el mensaje llega al servidor y este envía la mesa actualizada a todas. Cada laptop solo ve su propia mano; de las demás jugadoras solo se ve cuántas cartas tienen.

2. Tecnologías utilizadas

Tecnología	Uso en el proyecto
C#	Desarrollo de la aplicación gráfica y la lógica del juego.
Python y FastAPI	Creación de la API para comunicar la aplicación con la base de datos.
MySQL Workbench	Consulta y revisión de las tablas y los registros almacenados. Almacenamiento de jugadores, historial de partidas y movimientos del juego.
Git y GitHub	Control de versiones, respaldo del código y colaboración entre las integrantes.
Visual Studio y Visual Studio Code	Entornos utilizados para desarrollar la aplicación y la API.
Sockets TCP y JSON	Comunicación en red entre las laptops y el servidor del juego (puerto 5000); los mensajes se envían en formato JSON.

3. División de tareas entre las integrantes

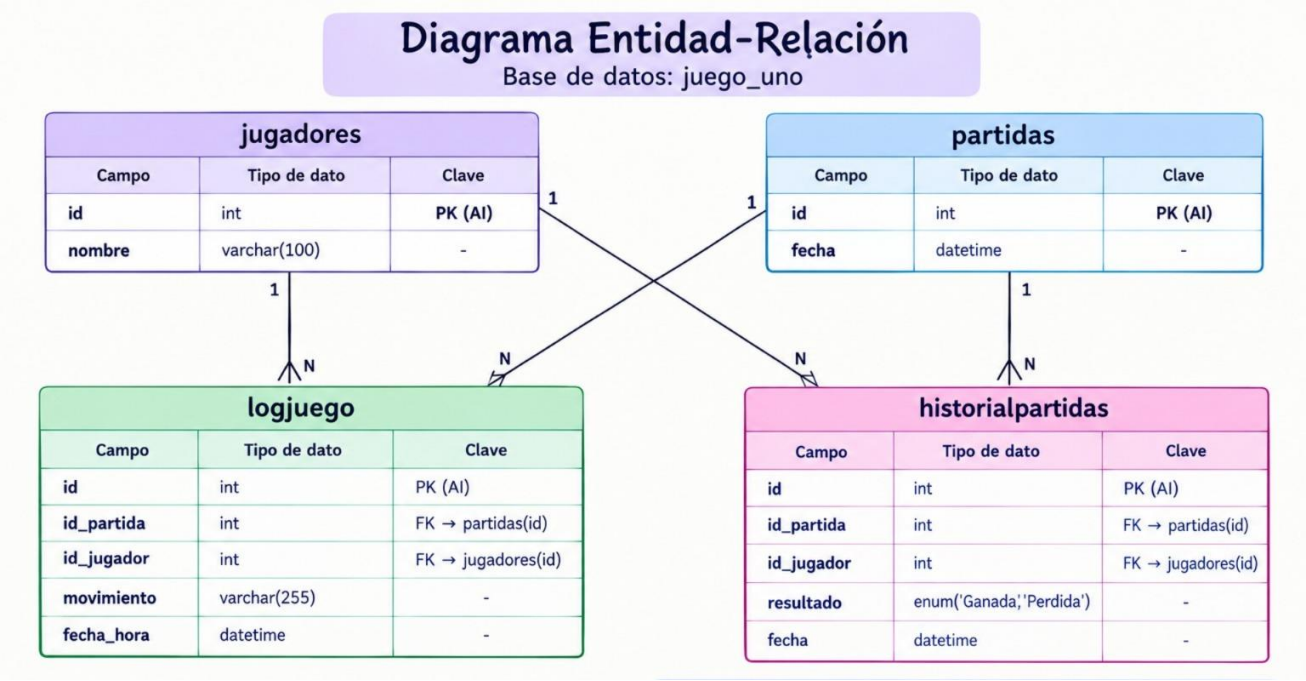
Gisela Rodríguez Gerónimo

- Creación y administración inicial del repositorio de GitHub.
- Implementación del registro de partidas, incluyendo el identificador y el resultado.
- Implementación del guardado del historial de partidas.
- Pruebas iniciales y avances de la interfaz gráfica en Windows Forms.
- Creación e implementación de la API con Python y FastAPI.
- Apoyo en la integración y corrección del bloqueo que ocurría al registrar movimientos mediante la API.
- Segunda etapa: configuración de la red entre las laptops (firewall y pruebas de ping).
- Segunda etapa: servidor de sockets TCP en Python y pantalla para conectarse por IP.
- Segunda etapa: reparto de cartas desde el servidor y vista de la mano de cada jugadora.
- Segunda etapa: robo de cartas y efectos de Salta, Reversa y +2 desde el servidor.
- Segunda etapa: temporizador de UNO controlado por el servidor, botón UNO y contador.

Alejandra Martínez Arévalo

- Pruebas iniciales del proyecto y limpieza de elementos duplicados.
- Ajustes para el funcionamiento de MySQL y pruebas para insertar y consultar jugadores.
- Incorporación del script de creación de la base de datos.
- Creación de las clases Carta, Mazo y Jugador.
- Implementación del reparto de cartas y visualización de la mano inicial.
- Implementación de efectos de cartas, robo de cartas, cancelación de turnos y reversa.
- Creación e implementación de las cartas y modificaciones de la interfaz, incluyendo fondo, botones y visualización de las cartas de ambos jugadores.
- Segunda etapa: sala de espera con registro de jugadoras en el servidor y botón Empezar.
- Segunda etapa: tirar carta, validación de la jugada en el servidor y cambio de turno.
- Segunda etapa: comodín y +4 con selector de color, y rebarajar el mazo cuando se acaba.
- Segunda etapa: fin de partida, historial en la base de datos, pantalla final y manejo de desconexiones.

4. Diagrama entidad-relación de la base de datos

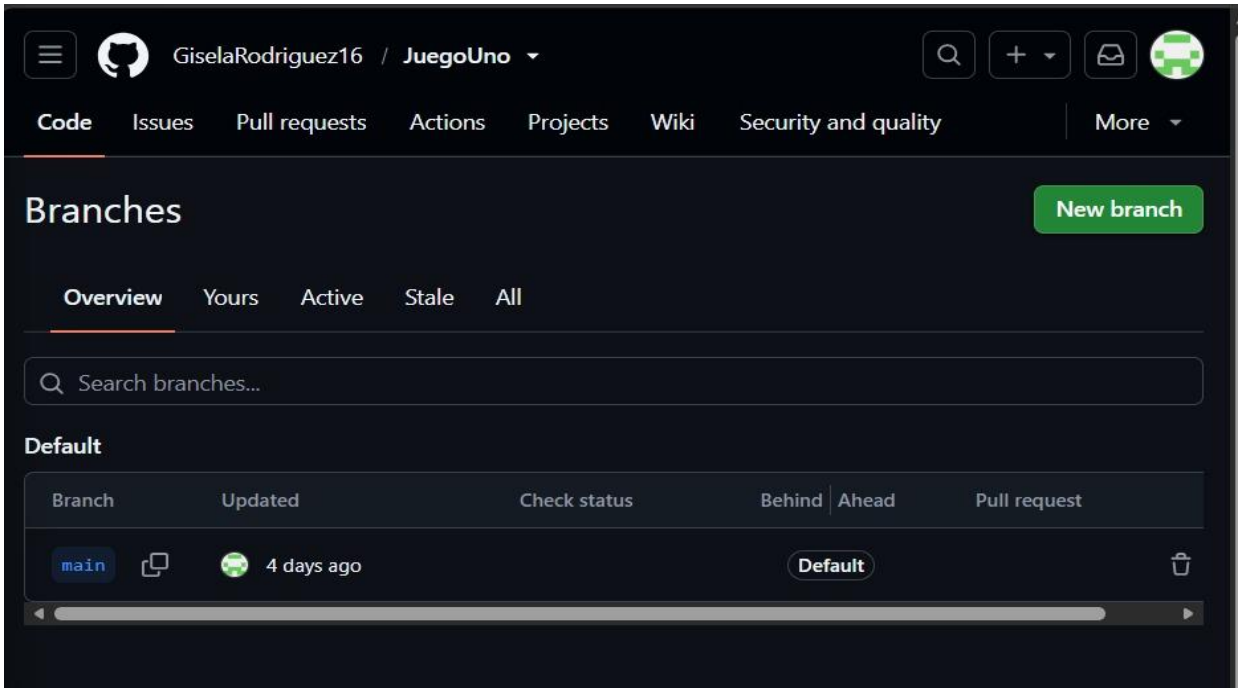


5. Listado de commits

Fecha	Integrante	Descripción
22/09/2026	Gisela	Creación del repositorio del proyecto.
22/09/2026	Alejandra	Primer commit de prueba inicial.
22/09/2026	Gisela	Commit de prueba.
22/09/2026	Alejandra	Limpieza de elementos duplicados.
22/09/2026	Alejandra	Ajustes menores para el funcionamiento de MySQL.
24/09/2026	Alejandra	Pruebas para insertar y consultar jugadores desde MySQL.
24/09/2026	Alejandra	Incorporación del script de la base de datos.
25/09/2026	Gisela	Registro de partidas, identificador de partida y resultado.
30/09/2026	Alejandra	Creación de las clases Carta y Mazo.

30/09/2026	Alejandra	Creación de Jugador, reparto de cartas y mano inicial.
30/09/2026	Alejandra	Efectos de cartas, robo, salto de turno y reversa.
01/10/2026	Gisela	Guardado del historial de partidas.
01/10/2026	Gisela	Prueba inicial de la ventana Windows Forms.
01/10/2026	Gisela	Avance de la interfaz gráfica.
05/10/2026	Alejandra	Creación e implementación de las cartas del juego.
06/10/2026	Alejandra	Cambios de interfaz: fondo, botones y cartas de ambos jugadores.
07/10/2026	Gisela	Creación e implementación de la API con Python y FastAPI.
11/10/2026	Gisela	Corrección del bloqueo al enviar movimientos a la API y actualización final.
01/11/2026	Gisela	Conexión en red: servidor de sockets TCP y pantalla para conectarse por IP.
02/11/2026	Alejandra	Sala de espera: registro de jugadoras en el servidor y botón Empezar.
03/11/2026	Gisela	El servidor baraja y reparte; cada laptop ve solo su mano.
04/11/2026	Alejandra	Tirar carta: el servidor valida la jugada, cambia el turno y actualiza a todas.
06/11/2026	Gisela	Robar carta y efectos de Salta, Reversa y +2 (completa partes del commit anterior).
06/11/2026	Alejandra	Comodín y +4 con elección de color, y rebarajar cuando se acaba el mazo.
07/11/2026	Gisela	UNO con temporizador en el servidor: botón UNO, contador y castigo.
08/11/2026	Alejandra	Fin de partida: ganadora, historial en la BD, pantalla final y desconexiones.

6. Listado de ramas generadas



7. Problemas encontrados y soluciones

Durante las primeras etapas fue necesario ajustar algunos elementos duplicados y realizar cambios para que la conexión con MySQL funcionara correctamente. Se hicieron pruebas de inserción y consulta de jugadores, además de incorporar el script de creación de la base de datos.

Al integrar la API con el juego, la aplicación se quedaba bloqueada al registrar movimientos. Se revisó el código de las llamadas a la API y se ajustó el manejo de las operaciones asíncronas, utilizando ConfigureAwait(false) en las llamadas esperadas. Después del cambio, el juego volvió a funcionar y se observaron respuestas HTTP 200 OK en las solicitudes de registro.

Y sobre el uso de github solo tuvimos que volver a familiarizarnos ya que no solemos usarlo mucho, ya lo habíamos usado en el pasado pero al principio nos confundiamos otra vez.

Segunda etapa: juego en red

Conexión entre las laptops. Al principio las computadoras no se hacían ping, aunque estaban en la misma red de la universidad. La causa era el firewall de Windows. Se crearon reglas con netsh para permitir el ping y los puertos 8000 (API) y 5000 (sockets).

Ping en un solo sentido. Después de crear las reglas, el ping solo funcionaba de una laptop a la otra. No afectó al juego, porque cada cliente abre la conexión hacia el servidor y las respuestas viajan por esa misma conexión.

Acceso a la API desde otra computadora. La API solo aceptaba conexiones de la misma máquina. Se solucionó ejecutando uvicorn con --host 0.0.0.0.

Imágenes de las cartas borradas. Al subir el proyecto se borraron las imágenes para reducir el peso, pero lo que realmente pesaba eran las carpetas bin, obj y .vs. Se recuperaron las imágenes, se configuró el proyecto para copiarlas al compilar y se agregó un archivo .gitignore.

Entorno virtual de Python. Al ejecutar el servidor aparecía el error "No module named dotenv" porque el entorno virtual no estaba activado y Windows no permitía ejecutar el script de activación. Se habilitó con Set-ExecutionPolicy y se activó el venv en cada terminal.

Mensajes en otro hilo. Los mensajes del servidor llegan en un hilo distinto al de la ventana, por lo que la interfaz no se actualizaba correctamente. Se usó BeginInvoke para pasar cada actualización al hilo de la ventana.

Cartas distintas en cada ventana. Mientras cada computadora barajaba su propio mazo, cada jugadora veía una partida diferente. Se movieron el mazo, el reparto y los turnos al servidor, y el juego solo dibuja lo que este le envía.

Temporizador de UNO. Si cada computadora cuenta su propio tiempo, el límite puede no coincidir. El temporizador se pasó al servidor y las laptops solo muestran el contador.

Errores al integrar los cambios. Hubo un error de sintaxis en servidor.py por la sangría de un bloque, que se corrigió en VS Code. También a un commit le faltaron algunas partes en EstadoJuego.cs y FormJuego.cs, que se completaron en el siguiente commit.

Pruebas con una sola computadora. Para probar en casa con dos jugadoras se abrió el ejecutable del juego dos veces y ambas ventanas se conectaron a 127.0.0.1.

Para trabajar entre las dos sin conflictos en GitHub, nos turnamos los cambios: antes de empezar, cada una hacía git pull, después subía su commit con git push y le avisaba a la otra.